Using spec-driven development with Claude Code

11 min read · 1 day ago

Heeki Park Follow

Listen Share ••• More

I no longer write code by hand. I wouldn't call myself a proper software development engineer by trade, nor have I deployed large-scale software systems. I am a solutions architect and have written mostly prototypes and small-scale applications. Still, those experiences, combined with reading, research, and schooling, have shaped my rudimentary software engineering approaches that I now take to AI tools like Claude Code.

I give this context because I want to say that my build approach isn't just giving into the vibes. There is a structured approach to documenting requirements, generating specifications, and reviewing the code that is generated. When deploying on AWS, there is design thinking to how resources are organized and deployed based on frequency of change.

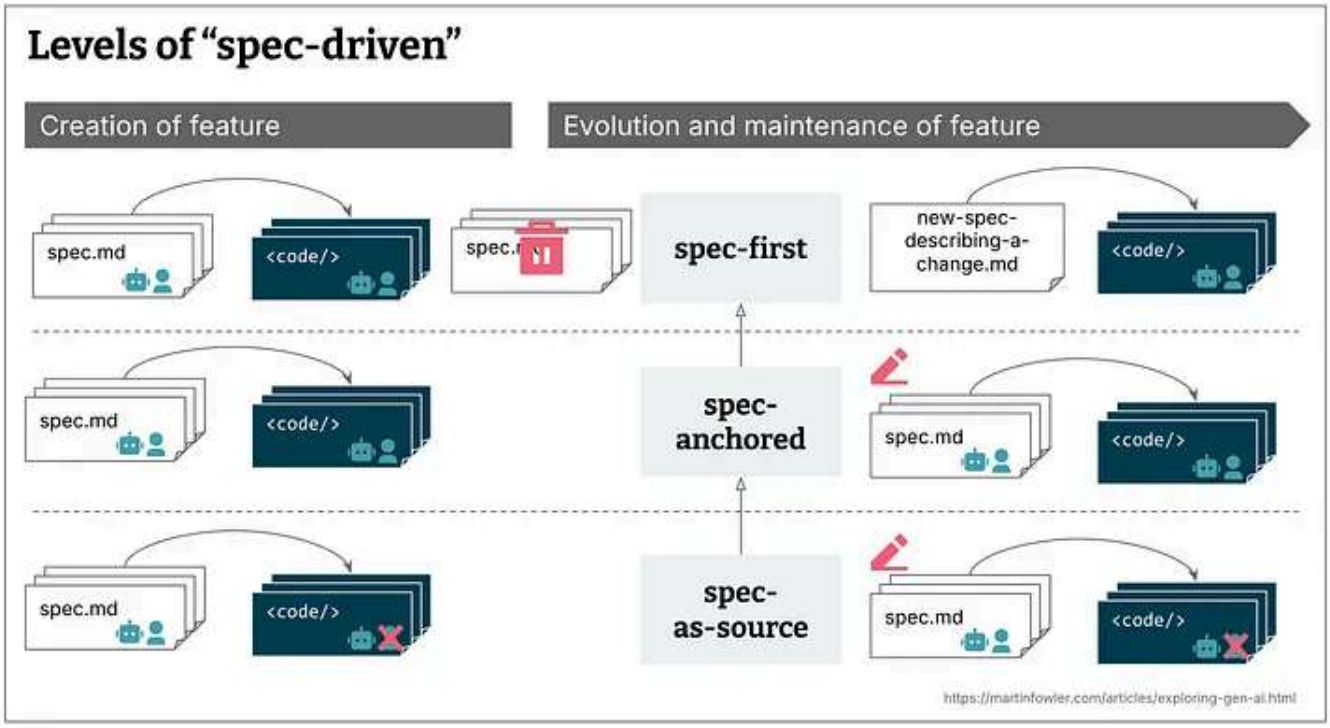
As such, I spend a lot of time in the planning phase of the project, rather than jumping right into code generation. Defining and documenting requirements early on leads to better outputs and a less frustrating overall experience. You might say this is true of any software engineering project before AI tools even hit the scene.

That said, the Pandora's box of AI tools has been opened, and now we need to pump the brakes on vibe coding before we add more tech debt to our backlog. Spec-driven development can act as those brakes, reinforcing good software engineering practices while also taking advantage of the productivity gains from generative AI.

Understanding spec-driven development

When I first started hearing about spec-driven development, I initially wondered if this was just about documenting the requirements first before proceeding with the implementation. Didn't we already have that all documented in Confluence, which we could then pull in via MCP? However, one also recognizes that even well-documented projects are subject to scope creep, feature divergence, gold plating, and tech debt from quick fixes.

Spec-driven development *can* help address some of those issues, but it depends on how it is applied. Birgitta Böckeler wrote an excellent blog post that outlines three levels of spec-driven development.

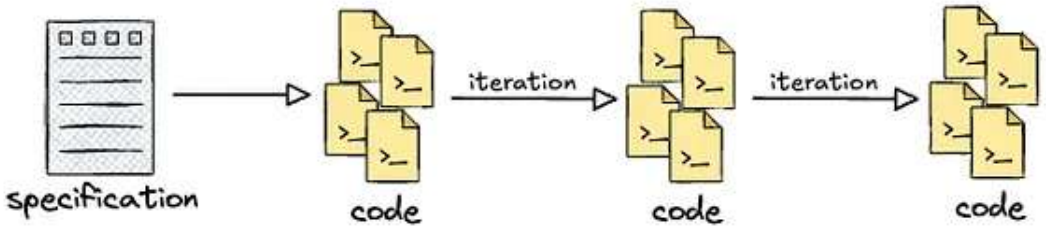


Levels of spec-driven development, as illustrated in the source blog post

1. spec-first: well thought-out spec is written first, then used in the AI-assisted development workflow
2. spec-anchored: spec is kept even after the task is complete, for evolution and maintenance of the respective feature
3. spec-as-source: spec is the main source file over time, only the spec is edited by the human, human never touches the code

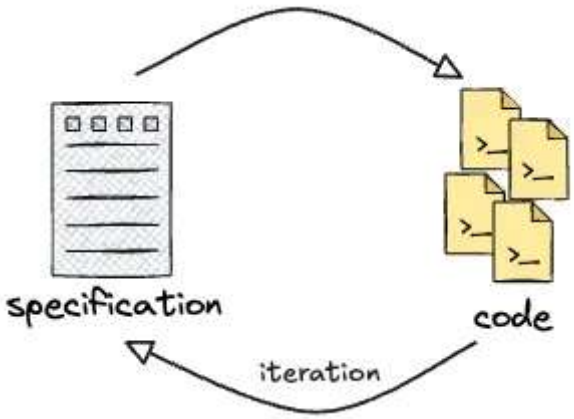
I think my general inclination is mostly toward spec-first approaches. It's very easy to start by iterating on a singular specification for the project and putting together a great design and implementation plan. It's equally easy to go off and start

implementing, leaving that one specification in the dust, especially as one gets deeper into the project. Perhaps that's not even spec-first but instead *spec-once* development.



Spec-once development, where the specification launches the project but is later forgotten

However, I think the value in spec-driven development is the fact that the developer is forced to think deeply about the use case requirements, document architectural considerations, and outline implementation approaches. As depicted in the source diagram, it also indicates that there could be many specification documents for different features. Furthermore, with the latter definitions of spec-driven, it forces the developer to constantly revisit those requirements and feed them back into the specification(s).



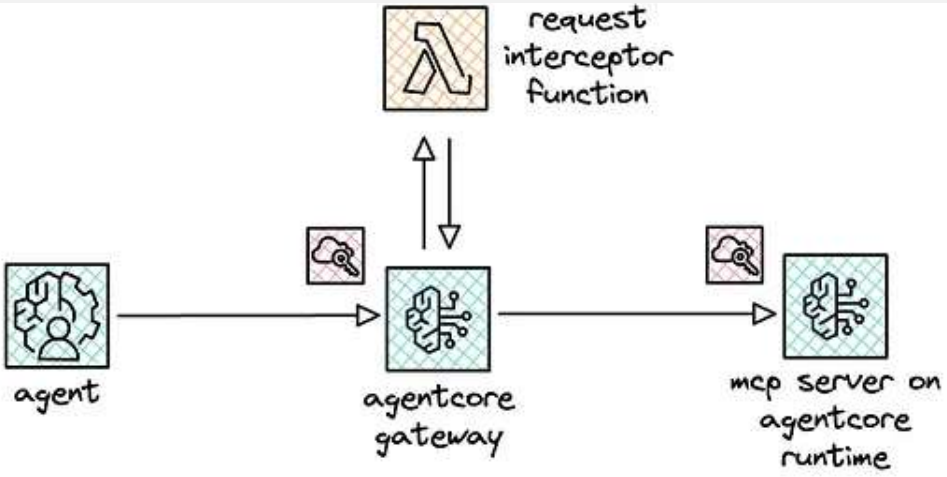
Spec-driven development as a continuous feedback loop

Planning and drafting the specification

For concrete practice, I wanted to better understand the nuts and bolts of how an interceptor worked with AgentCore Gateway. While documentation provides examples of input and output payloads with the interceptor function, I wanted to dig in firsthand.

Open in app ↗

Medium



Request interceptor flow with AgentCore Gateway

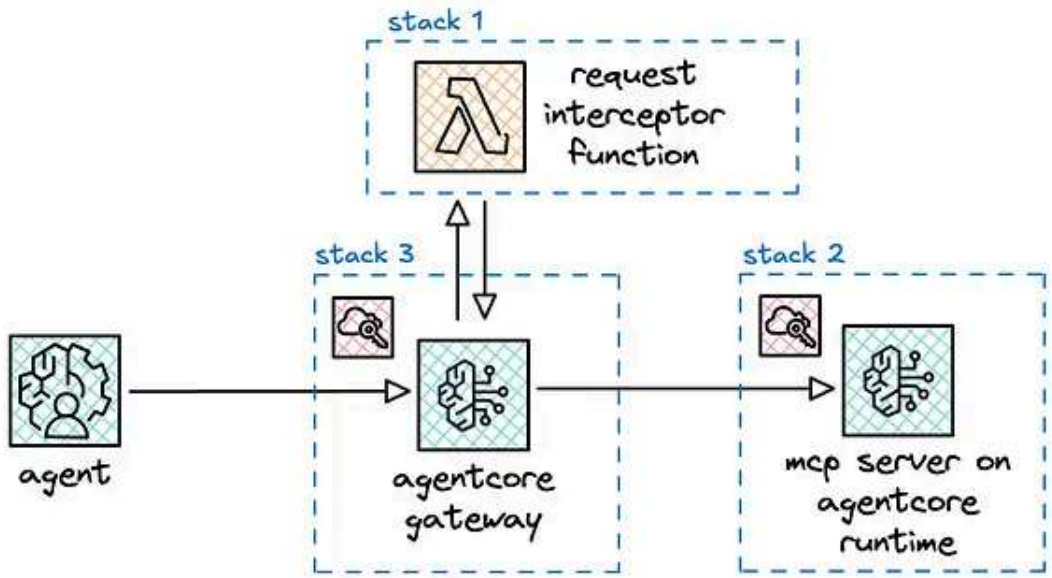
My normal flow for these prototyping endeavors is 1/ to conduct due diligence by reading documentation and other relevant sources, 2/ plan out the resources required for deployment in AWS, and 3/ start building small, testable modules. The research phase happens regardless, but I use those same phases when building out my initial prompt for Claude Code.

```
Claude Code v2.1.49
Opus 4.6 · Claude Pro
~/Documents/Isengard/github/agents/interceptors

I want to build a quick prototype for AgentCore Gateway that demonstrates the use of Lambda-based interceptors, which will be built in three separate phases.

Sub-project 1:
Create a sub-project for Lambda functions that will be used as a request interceptor for AgentCore Gateway.
Use SAM for infrastructure as code, SAM CLI for deployments, and a makefile for standardizing commands.

Phase 1: Create a Lambda function that will be used as a request interceptor for AgentCore Gateway.
Review documentation to ensure that the correct event payloads are used: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/gateway-interceptors.html
Review documentation to ensure that header keys are correct: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-header-allowlist.html. Headers should be prefixed with:
  X-Amzn-Bedrock-AgentCore-Runtime-Custom-
The request interceptor should add a custom header to the tool call.
Do a test invoke of the function with a sample payload to ensure that the function works correctly.
Document the deployed configurations in the root README.md file.

Sub-project 2:
Create a sub-project for AgentCore resources for the rest of the project.
Deploy AgentCore resources using the new AgentCore CLI where possible: https://github.com/aws/agentcore-cli
Do not use the boto3 SDK for deployments unless the new AgentCore CLI does not yet support the required integration.

Phase 1: Build a basic hello world MCP server on AgentCore Runtime.
The MCP server should log the event payload for debugging purposes.
Test the MCP server locally first, ensure that list tool and invoke tools work correctly.
Deploy the MCP server to AgentCore Runtime.
Test the MCP server that is deployed to AgentCore Runtime in the same way that it was tested locally.
Document the local and deployed configurations in the root README.md file.

Phase 2: Integrate the MCP server with AgentCore Gateway.
Create an AgentCore Gateway endpoint, using the new AgentCore CLI. Do not use the boto3 SDK for the deployment.
Add the MCP server on AgentCore Runtime as a target with no authorization configuration. Prefer the AgentCore CLI for creating this configuration, but if that configuration is not yet available,
create a boto3 script.
Test the MCP server that is behind AgentCore Gateway in the same way that it was previously tested.
Document the deployed configurations in the root README.md file.

Phase 3: Add the Lambda function request interceptor to AgentCore Gateway for tool calls.
Review documentation to ensure that the AgentCore Gateway service role has least privilege access to invoke the Lambda function request interceptor:
https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/gateway-interceptors-permissions.html
Integrate the Lambda function request interceptor from sub-project 1 with AgentCore Gateway.
Test the MCP server that is behind AgentCore Gateway in the same way that it was previously tested but check that the interceptor added the custom header.
Document the deployed configurations in the root README.md file.

Start by first creating a specification document and allow for iteration before starting the implementation.
Ask clarifying questions, as needed, using selectable inputs to make clarifying responses simple.
Write the specification as SPECIFICATIONS.md at the root of the project.

● Let me start by reviewing the existing project state and fetching the documentation you referenced.
```

Initial prompt for defining the project and build phases

1/ I read the CloudFormation documentation to understand the properties and parameters required to configure the relevant resources. I also directed Claude Code to go and ingest the documentation so that it operated on the same context and used the relevant resource properties.

2/ I spent time thinking through how I would build out the stack, considering resource dependencies, modular testing, and overall project flow. As such, I broke the overall project down into two sub-projects (red) with the appropriate phases (yellow) for each sub-project.



Modular stacks for simpler testing and composable deployment

For the first sub-project, I define a single phase (stack 1) for the interceptor. The interceptor is a Lambda function, which gets added to the Gateway resource via the `InterceptorConfigurations` property. I made this a separate stack so that I could test and iterate using SAM, as I do with all my serverless projects.

For the second sub-project, I define three phases to help me build stepwise. At the top level, I specify that my preference is the new AgentCore CLI, which is currently in public preview and aims to replace the starter toolkit. However, because it is in public preview, it doesn't yet support all the resources and configurations. In those scenarios, I direct it to not use the `boto3` SDK. In retrospect, I should have positively stated that it should then fall back to using CloudFormation. I point to CloudFormation documentation later throughout the instructions.

For the first phase (stack 2), I instruct it to create the MCP server and deploy it on AgentCore Runtime. I instruct it to test locally and then use the same test on the deployed endpoint in AgentCore Runtime.

For the second phase (stack 3), I instruct it to create the AgentCore Gateway resource and set up the MCP server in AgentCore Runtime as the target. I also set up a credential provider configuration to ensure that AgentCore Gateway uses OAuth2 when connecting to the MCP server. I instructed it to test the end-to-end stack to ensure that the integration worked properly.

For the third phase (stack 3 updates), I instruct it to integrate the interceptor at the Gateway, which injects a custom header that gets passed along to the MCP server. I added more logging at this phase, as I wanted to see the headers that were passed throughout the call stack and ultimately returned to the test client.

3/ Lastly, I made sure that there is plenty of iteration in a planning phase, writing all the plans as a specification. I spent time reviewing the specification to ensure that the design, architecture, and implementation plan aligned with my expectations.

Tidbits on working with Claude Code

I found that the default context window of 200k tokens was fine for my purposes. Context windows define how much information can be included in each round trip with Claude. The information includes relevant code, steering documents, available tools with their associated descriptions, data retrieved from tool calls, skills, conversation history, etc. The round trip includes both the input above and the response output.

The default context window limit on paid Claude plans is 200k tokens, which in my experience with the Pro subscription, also applies to Claude Code. The extended context window of 1m tokens is currently available only for users with the Max 20x subscription (womp womp).

What about if I swap from my Pro subscription to using Bedrock directly? I can swap by setting `CLAUDE_CODE_USE_BEDROCK=1` before starting up Claude Code. Bedrock has Claude Opus 4.6, Sonnet 4.6, Sonnet 4.5, and Sonnet 4, all of which support 1m token context windows in beta. In order to get access to that extended context window, I need to inject a custom header: `"anthropic-beta: context-1m-2025-08-07"`. However, Claude Code manages interactions with the API and as far as I am aware,

there is no way to add the beta flag. Hence, context window of 200k tokens for me it is.

```
/context
└ Context Usage
  us.anthropic.claude-opus-4-6-v1 · 156k/200k tokens (78%)

  Estimated usage by category
  System prompt: 2.7k tokens (1.4%)
  System tools: 16.2k tokens (8.1%)
  Memory files: 656 tokens (0.3%)
  Skills: 393 tokens (0.2%)
  Messages: 124.1k tokens (62.0%)
  Free space: 23k (11.5%)
  Autocompact buffer: 33k tokens (16.5%)

  Memory files · /memory
  └ CLAUDE.md: 656 tokens

  Skills · /skills
```

Viewing context usage in Claude Code

In heavy usage scenarios when I hit that context window limit, Claude Code automatically compacts the conversation history. I found that the compaction process took 3–5 minutes on the low end and 10–12 minutes on the high end.

I observed that Opus hits subscription plan usage limits really fast compared to Sonnet. With subscription plans, the usage limits are defined as the "conversation budget" which determines how many messages you can send. It is influenced by the conversation history, features used, and the chosen model. In my anecdotal experience with the Pro subscription, I ran into sliding window usage limits within 45 minutes to an hour of heavy Opus 4.6 use. When switching over to Sonnet 4.6, I did not hit any usage limits, even after a few hours of heavy usage.

I instruct Claude Code to ask clarifying questions, as needed, but also to use selectable inputs to make clarifying responses simpler. This way, it presents a set of options that I can then navigate via a menu of choices. It leaves the last option for open user input if needed, but I generally found that the presented options are sufficient. This makes the back and forth much easier and faster. At the end, I get a nice summary of the questions that it asked and the selected responses for each question.

```
←  ☒ Intercept scope  ☒ Interceptor CLI  ☒ IAM approach  ☒ AWS config  ✔ Submit  →

Review your answers

 ● For the interceptor Lambda output, the docs show headers can be added to 'transformedGatewayRequest.headers'. The current spec includes this. Should the interceptor ONLY add the header on 'tools/call'
 requests, or on ALL MCP requests (including 'tools/list', 'initialize', etc.)?
   → Only tools/call
 ● The AgentCore CLI supports 'agentcore create_mcp_gateway' with '--interceptor-configurations' for attaching interceptors at gateway creation time. Should we use that CLI command for Phase 3 instead of
 boto3?
   → CLI create_mcp_gateway
 ● For the gateway service role, should the SAM template in sub-project 1 also create the IAM role for AgentCore Gateway (with lambda:InvokeFunction and the trust policy for bedrock-agentcore.amazonaws.com),
 or should that be managed separately?
   → SAM template
 ● What AWS region and profile should be used for this prototype?
   → us-east-1 / default

Ready to submit your answers?

❯ 1. Submit answers
  2. Cancel
```

Summary of responses from Claude Code's set of clarifying questions

I found myself trusting the actions of Claude Code more and more over time. For each action that Claude Code takes, it generally presents three options: 1/ yes, 2/ yes and don't ask again for [this subset of actions], 3/ no. I initially started by mostly choosing yes and closely observing each action. Over time, I found many actions to be fairly innocuous, especially read-only actions where it was just fetching data. I even found myself auto-allowing writes to code files as I trusted the outputs more. That said, I still haven't gotten to the point of going full YOLO mode with the `--dangerously-skip-permissions` flag. I'll see where I land when I start experimenting with agent teams.

I enjoyed running multiple Claude Code sessions using `tmux`. I initially did this as I had one session with my subscription plan and two alternate sessions using Bedrock. However, I realized I didn't need to do that when I saw how easy it was to enable API access and resume the prior session.

```
/exit
export CLAUDE_CODE_USE_BEDROCK=1
claude --resume <session_id>
```

Regardless, multiple parallel sessions will be useful when experimenting with agent teams: `tmux -CC` in iTerm2 for the win!

I ran into this fun little nugget during this project. When Claude Code was trying to research an approach to solving a problem, it used web search to retrieve additional information, and it tried to fetch my own blog post!

```
● Fetch(https://aws.plainenglish.io/getting-started-with-bedrock-agentcore-runtime-3eaae1f517cc)
  └ Fetching…

Fetch

  https://aws.plainenglish.io/getting-started-with-bedrock-agentcore-runtime-3eaae1f517cc
  Claude wants to fetch content from aws.plainenglish.io

Do you want to allow Claude to fetch this content?
❯ 1. Yes
  2. Yes, and don't ask again for aws.plainenglish.io
  3. No, and tell Claude what to do differently (esc)
```

Claude Code pulling my own blog post, which was published with AWS in Plain English

Unfortunately, Claude Code failed to pull the post, as it seems like Medium blocks agents from retrieving content from its platform.

Lessons learned

Time spent in upfront planning pays dividends for implementation efficiency and output quality. In earlier projects, I would write a few quick sentences as part of the initial prompt and get generating as soon as possible. I found myself course correcting a lot and occasionally wondering if I needed to start from scratch. While I didn't one-shot this project, I found most of my follow-up interactions were small tweaks rather than wholesale changes to the entire project. Furthermore, I frequently took learnings from the project and integrated them into my `CLAUDE.md` steering document and even ended up creating a `SKILL.md` at the end of the project. I think custom skills creation will likely be a significant accelerator for future projects.

Build stepwise in small, easily testable chunks. The upfront planning makes it easier to test and correct functionality. This article discusses approaches for building an elite engineering culture. It also covers the use of stacked pull requests, which creates smaller, more easily reviewable pull requests, each of which have dependencies on each other.

Flexibility and workarounds are often required. I really like the new AgentCore CLI. However, as it's still in public preview at the time of this writing, none of the functionality that I needed for implementation was supported in the new CLI. As such, I ended up going the CloudFormation route. I also found that I couldn't create credential providers using CloudFormation and had to use `boto3` to create it. From there, I was able to inject the provider ARN as a property for the Gateway target.

Build security in sooner than later. Obvious, I know. I added the last layer of OAuth 2 for ingress to the Gateway at the end, and it resulted in having to redeploy the entire stack. I found that the Gateway `AuthorizerType` property cannot be changed in place and instead requires the resource to be replaced entirely. However, because I had to create the credential provider via `boto3` it required a multi-step delete and recreate process. Fortunately, Claude Code handled this with relative ease. However, in enterprise deployment pipelines, this might be a tad bit more painful.

Regularly revisit and update documentation. At each step, if I needed to course correct and tweak the design, I made sure that I updated the specification. While I conjecture that I mainly applied a spec-first approach, I continually revisited the specification and tried to move closer to spec-anchored. It made sure that I stayed grounded, revisiting my original intent and modifying as appropriate.

Conclusion

This space is moving so fast. I have been spending more and more time in Claude Code and figuring out how to use the latest capabilities and techniques. I encourage you to think of your own small project idea and get hands on, regardless of your background, because with these tools, anyone can build.

The frontier keeps charging ahead. I have another bigger project idea in mind, where I aim to apply agent teams for the build. This project likely has far more than I can interactively build, so I want to explore how I might do this with agent teammates to whom I assign issues from a backlog. I plan to document that journey in an upcoming blog post. Stay tuned!

Resources

- https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html
- https://github.com/heeki/agents/tree/main/interceptors
- https://x.com/bcherny/status/2017742741636321619
- https://www.cjroth.com/blog/2026-02-18-building-an-elite-engineering-culture
- https://builder.aws.com/content/39vdJOeWD0pbA7hpKpW1a2tBw8x/extending-claude-code-with-plugins-and-skills-for-aws-development

Agentic Ai | Genai | Claude Code | AWS | Amazon Bedrock Agentcore

Follow

Written by Heeki Park

1.1K followers · 28 following

Principal Solutions Architect @ AWS. Opinions are my own. https://linktr.ee/heekipark

Responses (2)

Holger Kreissl

What are your thoughts?

Juan Cordero
19 hours ago

Great article, cleared up some things for me on the agentic side a bit. Have you written a deep dive article into agentic agent deployment uses yet? If not that would be great read as well.

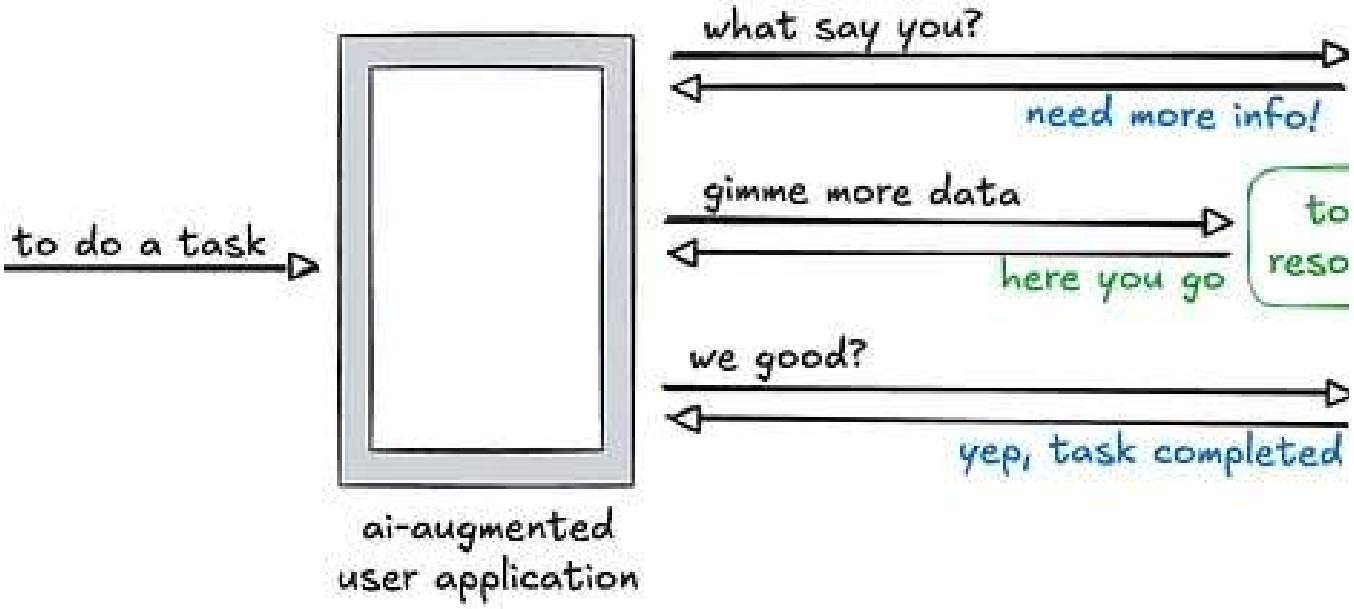
Reply

Darryl Ruggles
1 day ago

I have yet to really use spec-driven development with Claude Code but will try out this approach. I am now totally sold on using these tools though and feel very confident using them as an experienced developer who knows when to push back when it is going down the wrong path. We live in amazing times!

1 reply Reply

More from Heeki Park



Heeki Park

Building an MCP server as an API developer

Anthropic released MCP at the end of November 2024. It took a few months for it to catch on, but my, oh my, it feels like the community is...

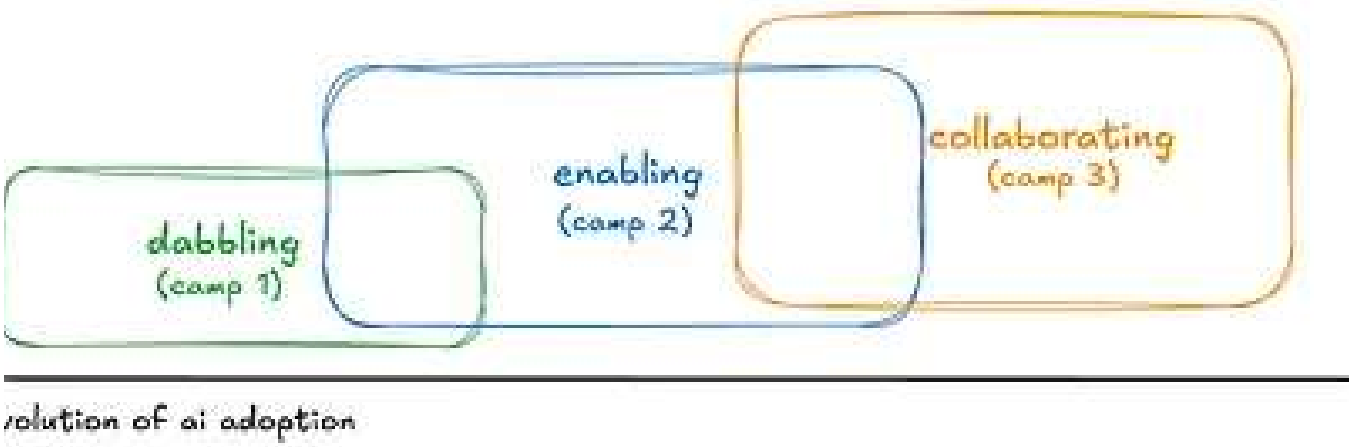
May 14, 2025 592 6

Heeki Park

Understanding OAuth2 and implementing identity-aware MCP servers

As excitement with MCP continues to build, folks are looking at how to leverage MCP and are considering deployment patterns for their...

Jun 6, 2025 412 5

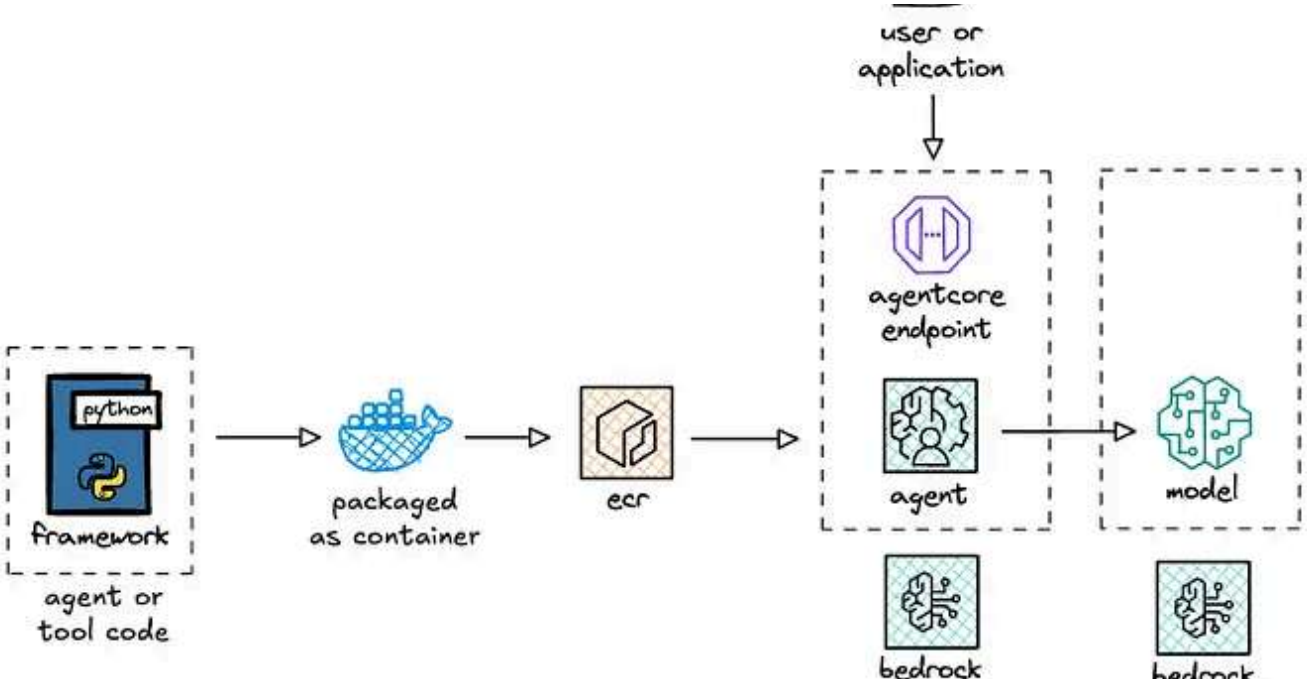


Heeki Park

Considering camps of adoption for the agentic journey

In my last post, I documented 10 observations with regard to agents. In this post, I reflect on learnings from conversations with...

Jan 30 68 1



In AWS in Plain English by Heeki Park

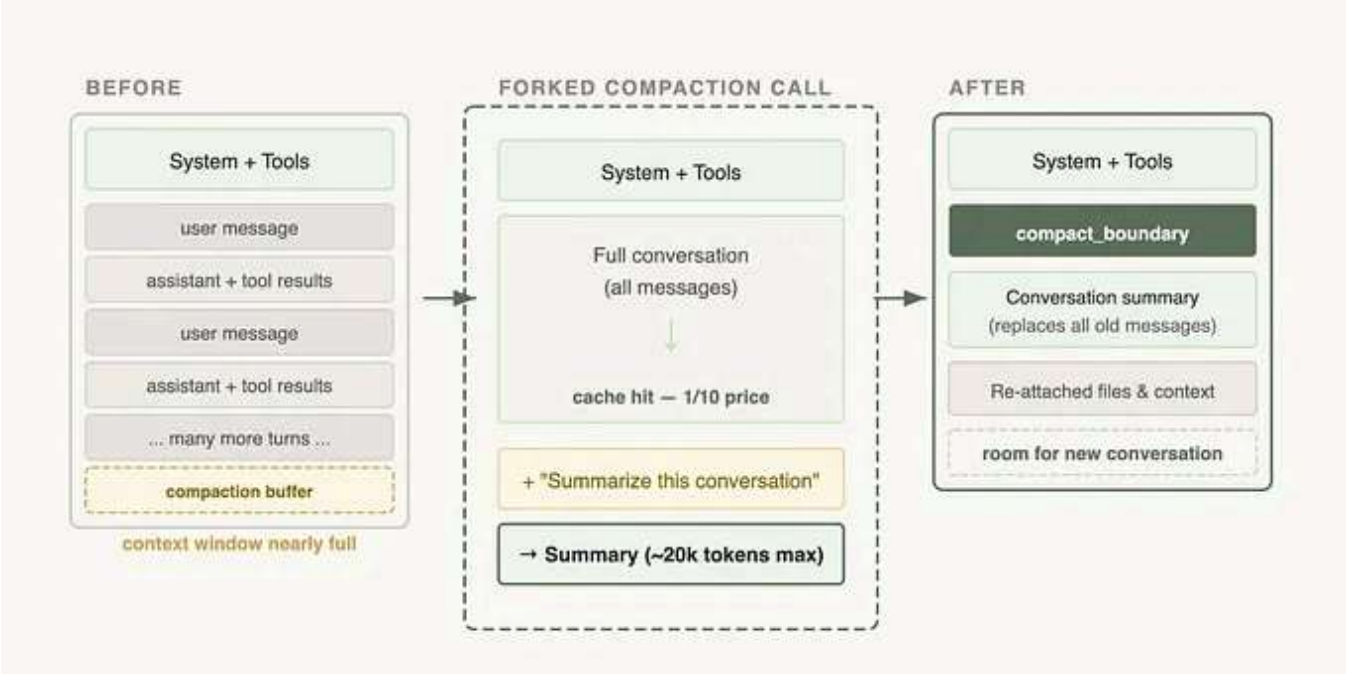
Getting started with Bedrock AgentCore Runtime

At the AWS Summit in New York this year, Amazon announced a new set of services for building, deploying, and operating your AI agents under...

Aug 13, 2025 109

See all from Heeki Park

Recommended from Medium

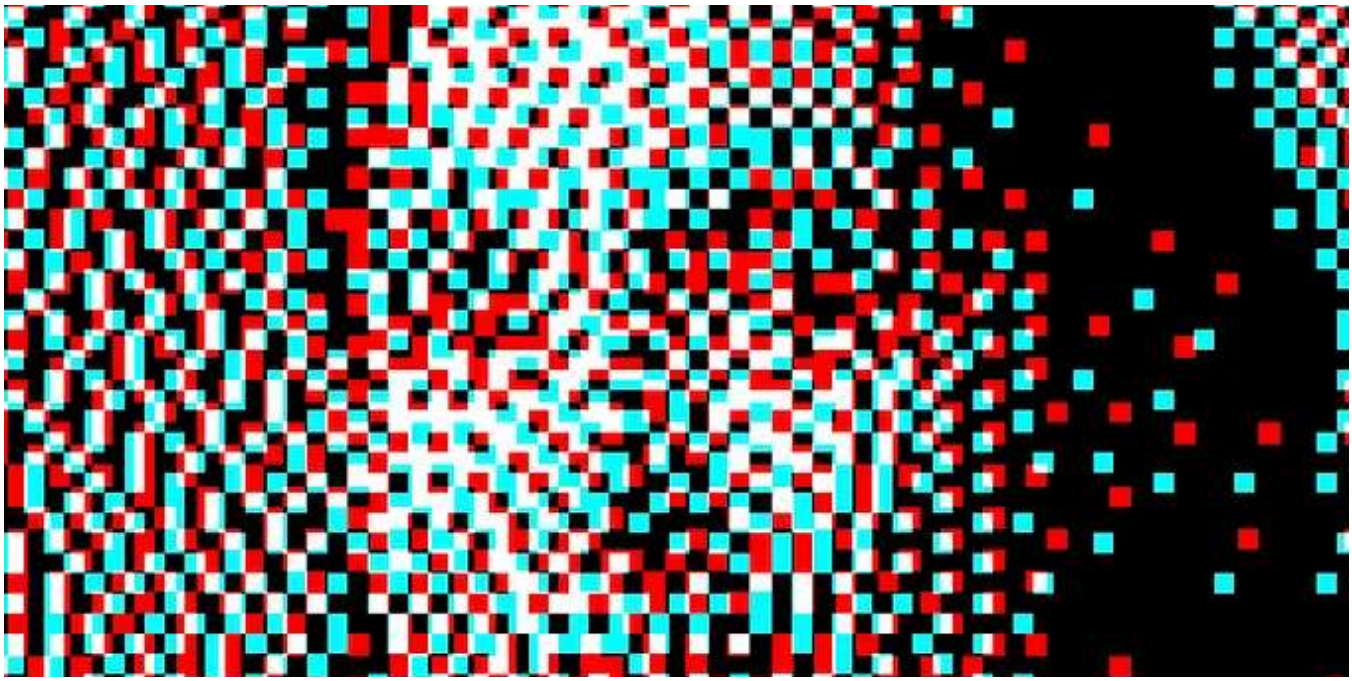


In AI Software Engineer by Joe Njenga

Anthropic Just Fixed the Biggest Hidden Cost in AI Agents (Automatic Prompt Caching)

With just one change, you can cut Claude API costs to 10 cents on the dollar and stop bleeding cash on every single API call.

Feb 20 729 16

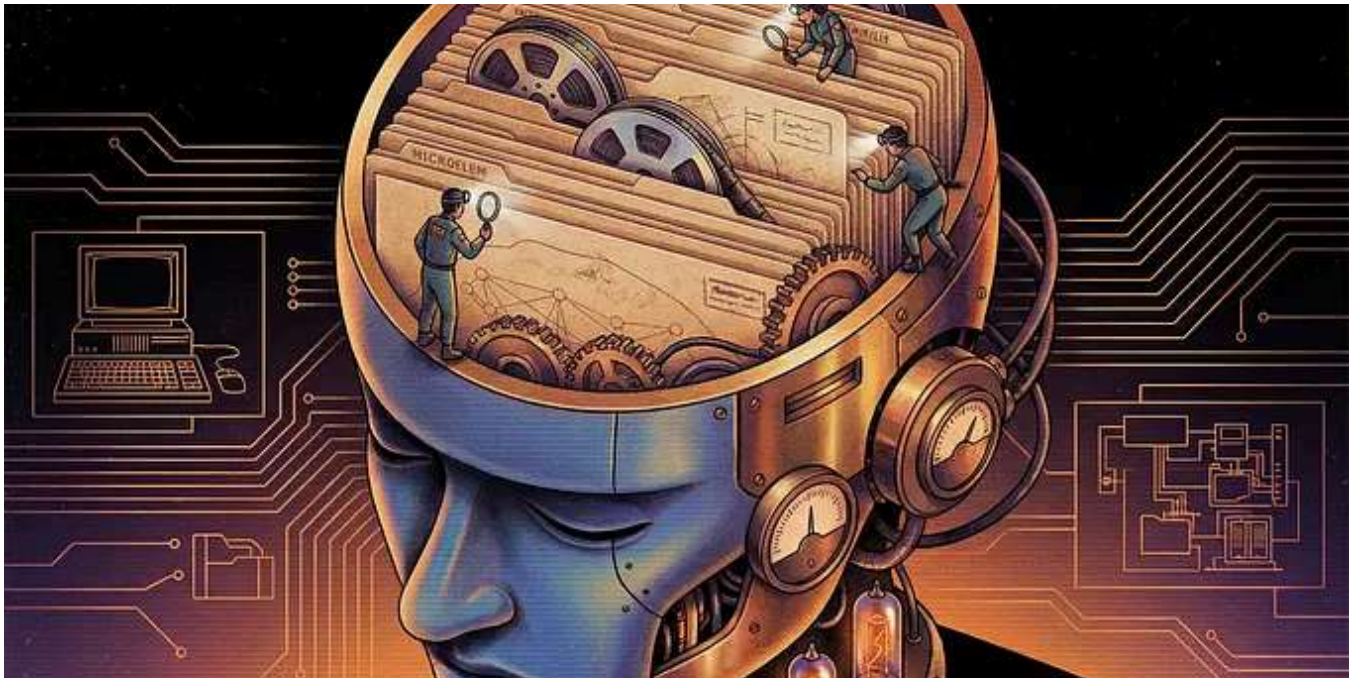


Marco Kotrotsos

The Agentic Engineering Playbook.

According to OpenClaw's Creator

Feb 17 210 8

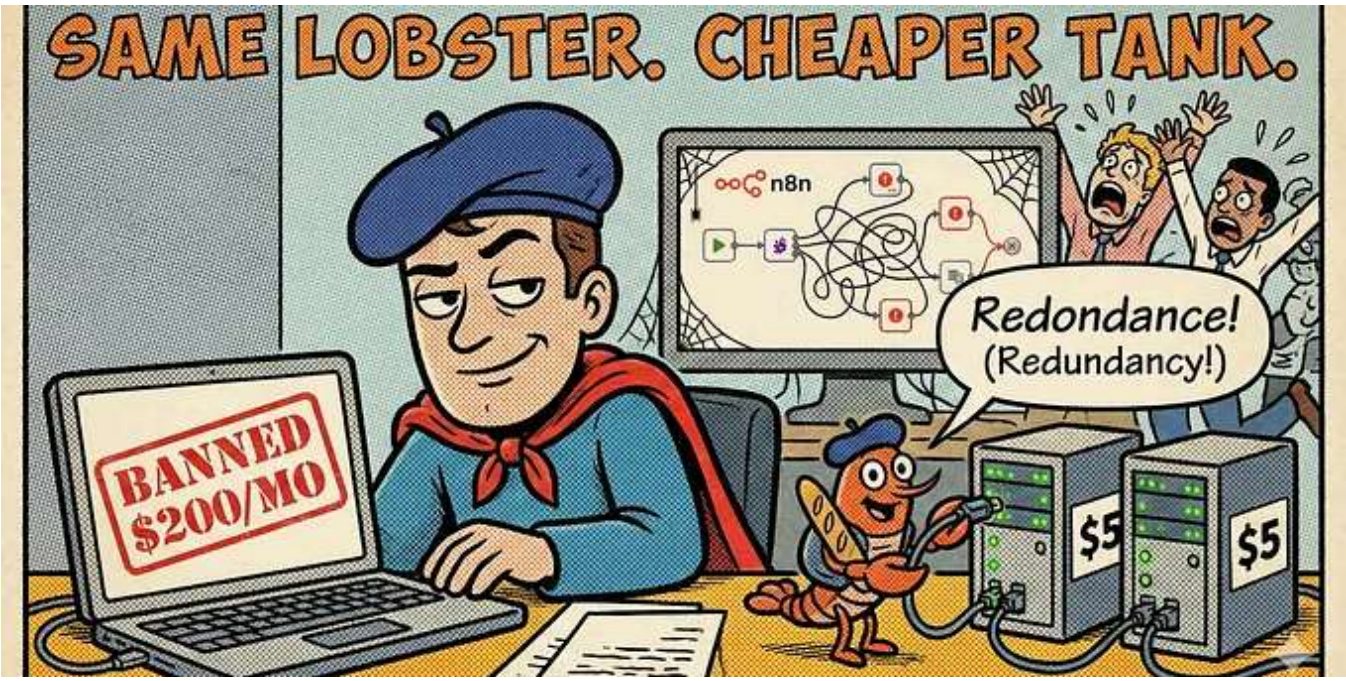


Agent Native

OpenClaw Memory Systems That Don't Forget: QMD, Mem0, Cognee, Obsidian

If your agent has ever randomly ignored a decision you know you told it... it's not random.

Feb 20 132 2

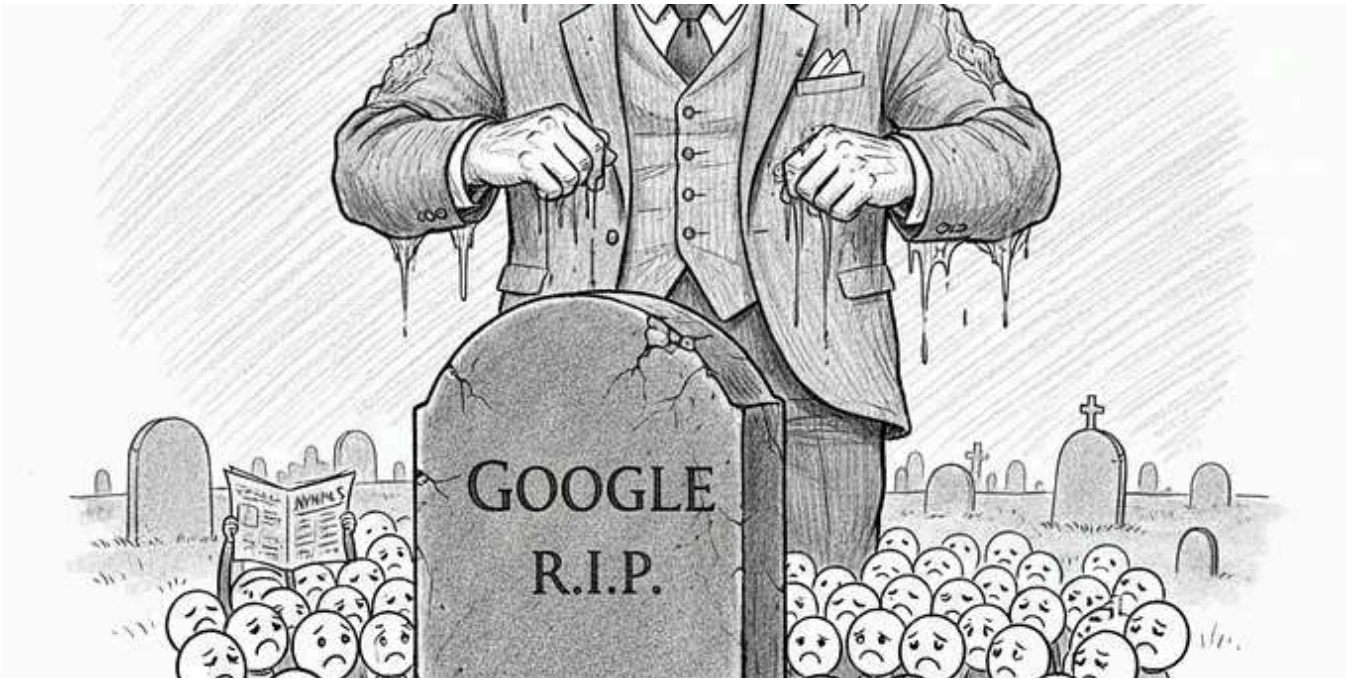


Phil | Rentier Digital Automation

Anthropic Just Killed My $200/Month OpenClaw Setup. So I Rebuilt It for $15.

My OpenClaw instance ran on Claude Max for six weeks. Emails, calendar, Telegram, the whole personal Jarvis fantasy. Then one morning...

Feb 19 765 28



In Level Up Coding by Teja Kusireddy

Google Is Quietly Dismantling Everything OpenAI Built

The most dangerous failure in Silicon Valley isn't bankruptcy. It's becoming the engine inside someone else's car.

Feb 17 4K 136

Rick Hightower

What Is GSD? Spec-Driven Development Without the Ceremony

How AI agents forget what they're building , and how to fix it

Feb 23 69

See more recommendations